

Resolución de la Actividad 9: Clase 05

Grupo 4

Integrantes:

- Avila, Aldana, legajo 1193721, alavila@uade.edu.ar
- Caivano, Alexander Francisco, legajo 1209389, acaivano@uade.edu.ar
- Casareski, Juan Ignacio, legajo 1176109, jcasareski@uade.edu.ar
- Segura, Juan Ignacio, legajo 1242159, jsegura@uade.edu.ar

Parte A: selección del subdominio

Se elige el subdominio de definición de procesos: la definición publicada de un proceso, sus pasos, las transiciones entre pasos, las reglas que se evalúan y los campos de formulario. Como caso de referencia se usa el proceso de alta de proveedor del tenant empresa_acme, trabajado en las actividades anteriores.

1. ¿Qué subdominio eligieron?
Definición de procesos: ProcessDefinition, Step, Transition, Rule y FormField.
2. ¿Por qué ese subdominio tiene objetos complejos?
Una definición de proceso combina identidad y ciclo de vida propio (versiones, estados draft, published, deprecated), una estructura interna compuesta (pasos de distinto tipo, transiciones con condiciones, formularios con campos) y comportamiento asociado (validar el grafo antes de publicar, evaluar reglas, validar formularios). Esa mezcla de estado, composición y lógica es más de lo que expresa un documento plano.
3. ¿Qué comportamiento tendría sentido encapsular en métodos?
La publicación de una versión (Publish), la validación estructural del flujo (ValidateGraph: existe inicio y fin, todos los pasos son alcanzables), la evaluación de una regla sobre los datos de la instancia (Evaluate) y la validación de los datos cargados contra los campos del formulario (ValidateInput).
4. ¿Qué validaciones pertenecen al objeto?
Las que dependen solo del estado del propio objeto: que el grafo tenga un paso inicial y al menos uno final, que no queden pasos inalcanzables, que toda transición apunte a un paso existente, que cada campo requerido tenga tipo definido y que no se publique una versión ya publicada.
5. ¿Qué parte seguiría siendo mejor resolver con MongoDB, Cassandra, Neo4j o Redis?
El almacenamiento operativo de definiciones e instancias queda en MongoDB, el historial de eventos en Cassandra, las consultas de caminos y dependencias entre pasos en Neo4j (como en la práctica 7) y el caché de definiciones publicadas, sesiones y colas en Redis. IRIS se evalúa solo para modelar la definición como objeto con comportamiento propio.

Parte B: identificación de clases

Clase	Tipo	Descripción	¿Persistente o embebida?	Justificación
ProcessDefinition	Clase persistente	Definición de un proceso, con versión y estado de publicación	Persistente	Tiene identidad, versionado y ciclo de vida propio (draft, published, deprecated)
Step	Clase base	Nodo del flujo, padre de todos los tipos de paso	Persistente	Concentra lo común a todo paso y se referencia desde las transiciones

Clase	Tipo	Descripción	¿Persistente o embebida?	Justificación
FormStep, DecisionStep, HumanTaskStep, ApiCallStep, NotificationStep, EndStep	Clases especializadas	Tipos concretos de paso, cada uno con datos y comportamiento propios	Persistentes	Heredan de Step y redefinen su ejecución y su validación
Transition	Clase de relación	Conexión entre dos pasos, con condición y prioridad	Persistente	Relaciona dos pasos y carga datos propios sobre la relación
Rule	Clase persistente	Regla de negocio evaluable sobre los datos de la instancia	Persistente	Se reutiliza en varios procesos del mismo tenant; necesita identidad para versionarla y corregirla en un solo lugar
FormField	Objeto embebido	Campo de un formulario (nombre, tipo, requerido)	Embebida	No tiene sentido fuera de su FormStep; se carga y se guarda con él
SLA	Objeto embebido	Plazo de resolución de un paso humano	Embebida	Es un valor del paso, sin identidad ni consultas propias
RetryPolicy	Objeto embebido	Política de reintentos de una llamada externa	Embebida	Solo existe dentro de su ApiCallStep

Parte C: propiedades y métodos

Clase	Propiedades principales	Métodos propuestos	Validaciones
ProcessDefinition	TenantId, ProcessId, Version, Name, Status, Steps	Publish(), ValidateGraph(), NewVersion()	Existe inicio y fin, pasos alcanzables, no publicar dos veces la misma versión
Step (base)	StepId, Name, Position, Definition	CanExecute(context), Describe()	StepId único dentro de la definición
FormStep	Fields (lista de FormField)	ValidateInput(data)	Cada campo requerido presente y con el tipo declarado
DecisionStep	Rules, transiciones de salida	Decide(data)	Una transición de salida por cada resultado posible
HumanTaskStep	Role, Sla	AssignTo(role), IsExpired(now)	El rol existe en el tenant; SLA mayor a cero
ApiCallStep	Endpoint, Method, Retry	Invoke(payload), ShouldRetry(attempt)	Endpoint con formato de URL; reintentos acotados
Rule	RuleId, Expression, OnResult	Evaluate(data)	Expresión parseable antes de guardar

1. ¿Qué método expresa una regla real del negocio?
Rule.Evaluate(data): decide, por ejemplo, si la documentación del proveedor está completa, que es la regla que bifurca el flujo de alta de proveedor.
2. ¿Qué método evita duplicar lógica en distintas partes del sistema?
ProcessDefinition.ValidateGraph(): la validación estructural del flujo se escribe una vez en la clase y la usan el editor de procesos, la publicación y la importación de definiciones.
3. ¿Qué validación debe ejecutarse antes de persistir el objeto?
La de consistencia interna de la definición: StepId únicos, transiciones que apuntan a pasos existentes y campos de formulario con tipo definido. Si se guarda una definición rota, fallan todas las instancias que se creen a partir de ella.
4. ¿Qué método no corresponde al objeto y debería estar en un servicio?
ApiCallStep.Invoke(payload): ejecutar la llamada HTTP real involucra credenciales, red y reintentos en tiempo de ejecución. El paso describe la llamada; un servicio de integraciones la ejecuta.

Parte D: objetos embebidos

Objeto embebido	Objeto contenedor	Propiedades	Motivo
FormField	FormStep	Name, Label, Type, Required, Options	No existe fuera de su formulario; se lee y se escribe junto con el paso
SLA	HumanTaskStep	Hours, WarningThreshold, OnExpire	Es un dato del paso; nunca se consulta solo
RetryPolicy	ApiCallStep	MaxAttempts, BackoffSeconds, OnFailure	Configura únicamente a su llamada
Condition	Transition	Expression, Priority	Solo tiene sentido evaluada sobre su transición

1. ¿Qué objeto embebido no tendría sentido consultar de manera independiente?
SLA: un plazo sin saber de qué paso es no responde ninguna pregunta. Siempre se llega a él a través del paso.
2. ¿Qué objeto embebido podría crecer demasiado si se modela mal?
FormField: un formulario con decenas de campos, cada uno con opciones, validaciones y textos por idioma, infla el paso que lo contiene. Si los campos se comparten entre formularios conviene moverlos a un catálogo persistente y referenciarlos.
3. ¿Qué objeto debería pasar a ser persistente si se reutiliza en muchos procesos?
La regla de validación. Si la condición de documentación completa se usa en varios procesos del tenant, conviene la clase Rule persistente y referenciada, para corregirla en un solo lugar. Por eso en la parte B ya se la definió como persistente.

Parte E: herencia y especialización

Jerarquía propuesta para los pasos del flujo:

```

Step
├── StartStep
├── FormStep
├── DecisionStep
├── HumanTaskStep
├── ApiCallStep
└── NotificationStep

```

└ EndStep

Clase base	Clases derivadas	Qué comparten	Qué cambia
Step	StartStep, FormStep, DecisionStep, HumanTaskStep, ApiCallStep, NotificationStep, EndStep	StepId, Name, Position, referencia a su definición, CanExecute(context), Describe()	Las propiedades específicas (Fields, Role y Sla, Endpoint y Retry) y la implementación de la ejecución y la validación

1. ¿Qué propiedades comparten todos los tipos de paso?
StepId, Name, Position dentro del flujo y la referencia a su ProcessDefinition.
2. ¿Qué método podría ser común a todos?
CanExecute(context): todo paso decide si puede ejecutarse con los datos actuales de la instancia.
3. ¿Qué método debería redefinirse según el tipo de paso?
La implementación de CanExecute y la validación previa al guardado: el formulario valida campos, la decisión exige reglas y transiciones de salida, la tarea humana exige rol y SLA, la llamada externa exige endpoint y reintentos acotados.
4. ¿Qué riesgo aparece si fuerzan herencia donde bastaba composición?
Jerarquías artificiales que se rompen con la primera variante nueva. Por ejemplo, un ApprovalStep que hereda de HumanTaskStep solo para sumar un campo de decisión: bastaba un HumanTaskStep con un FormField de decisión. La herencia obliga a tocar la jerarquía por cada variante; la composición la resuelve con datos.

Parte F: relaciones entre objetos

Clase origen	Relación	Clase destino	Cardinalidad	Justificación
Tenant	posee	ProcessDefinition	1 a N	Cada definición vive dentro de un tenant
ProcessDefinition	compone	Step	1 a N	Los pasos existen dentro de una versión de la definición
Step	sale por	Transition	1 a N	Un paso puede tener varias salidas condicionales
Transition	llega a	Step	N a 1	Cada transición apunta a un único paso destino
DecisionStep	evalúa	Rule	N a N	Una decisión evalúa varias reglas y una regla se reutiliza en varias decisiones
HumanTaskStep	se asigna a	Role	N a 1	Un paso se asigna a un rol y un rol atiende muchos pasos

1. ¿Qué relaciones deberían ser navegables desde el objeto?
ProcessDefinition hacia sus Steps y Step hacia sus Transitions de salida: validar y recorrer el flujo necesita esa navegación directa.
2. ¿Qué relaciones no conviene cargar automáticamente?
Las inversas masivas: Tenant hacia todas sus definiciones y Role hacia todos los pasos que tiene asignados. Se consultan con SQL cuando hacen falta.

3. ¿Qué relación podría generar acoplamiento excesivo?
Que Step navegue hacia las instancias en ejecución (ProcessInstance o ExecutionContext). La definición no debe conocer a sus instancias: cualquier cambio en la ejecución arrastraría a la definición.
4. ¿Qué relación ya está mejor resuelta en Neo4j?
Los recorridos sobre las transiciones: detección de ciclos, alcanzabilidad de pasos y pasos que comparten rol o API entre procesos. En la práctica 7 esas consultas se resolvieron con caminos de profundidad variable, algo que en un modelo de objetos exige recursión manual.

Parte G: representación conceptual estilo IRIS

```

Class FlowOps.ProcessDefinition Extends %Persistent
{
    Property TenantId As %String [ Required ];
    Property ProcessId As %String [ Required ];
    Property Version As %Integer [ InitialExpression = 1 ];
    Property Name As %String;
    Property Status As %String(VALUELIST = ",draft,published,deprecated");
    Relationship Steps As FlowOps.Step [ Cardinality = many, Inverse = Definition ];

    Method ValidateGraph() As %Boolean
    {
        // Existe un StartStep y al menos un EndStep
        // Todos los pasos son alcanzables desde el inicio
        // Toda transicion apunta a un paso existente
    }

    Method Publish() As %Status
    {
        // Falla si Status ya es published
        // Llama a ValidateGraph(); si falla, no publica
        // Cambia Status a published
    }
}

Class FlowOps.Step Extends %Persistent
{
    Property StepId As %String [ Required ];
    Property Name As %String;
    Property Position As %Integer;
    Relationship Definition As FlowOps.ProcessDefinition [ Cardinality = one, Inverse = Steps ];

    Method CanExecute(context) As %Boolean
    {
        // Cada subclase redefine su condicion de ejecucion
    }
}

Class FlowOps.HumanTaskStep Extends FlowOps.Step
{
    Property Role As %String [ Required ];
    Property Sla As FlowOps.SLA;

    Method IsExpired(now As %TimeStamp) As %Boolean
    {
        // Compara now contra la creacion de la tarea mas Sla.Hours
    }
}

Class FlowOps.SLA Extends %SerialObject
{
    Property Hours As %Integer [ Required ];
    Property WarningThreshold As %Integer;
    Property OnExpire As %String(VALUELIST = ",escalar,reasignar,notificar");
}

```

Parte H: SQL sobre objetos

IRIS proyecta cada clase persistente como una tabla: las subclases exponen sus propiedades más las heredadas, y los objetos embebidos se proyectan como columnas (Sla_Hours).

¿Qué definiciones publicadas tiene un tenant?

```
SELECT ProcessId, Version, Name
FROM FlowOps.ProcessDefinition
WHERE TenantId = 'empresa_acme'
AND Status = 'published';
```

Para qué sirve: listar los procesos disponibles para crear instancias en el tenant.

¿Qué pasos humanos tienen un SLA mayor a 24 horas?

```
SELECT d.ProcessId, s.StepId, s.Name, s.Sla_Hours
FROM FlowOps.HumanTaskStep s
JOIN FlowOps.ProcessDefinition d ON s.Definition = d.ID
WHERE d.TenantId = 'empresa_acme'
AND s.Sla_Hours > 24;
```

Para qué sirve: detectar pasos con plazos largos antes de publicar una versión.

¿Qué roles concentran más pasos asignados?

```
SELECT s.Role, COUNT(*) AS pasos_asignados
FROM FlowOps.HumanTaskStep s
JOIN FlowOps.ProcessDefinition d ON s.Definition = d.ID
WHERE d.TenantId = 'empresa_acme'
AND d.Status = 'published'
GROUP BY s.Role
ORDER BY pasos_asignados DESC;
```

Para qué sirve: ver la carga de trabajo por rol al diseñar o revisar procesos.

Parte I: decisión técnica

Criterio	¿IRIS aporta valor?	Justificación
Objetos con comportamiento	Sí	Métodos como Publish() o Evaluate() viven junto a los datos que validan
Herencia y especialización	Sí	La jerarquía de Step se declara con Extends en lugar de simularse con un campo type
Validaciones complejas	Sí	Se ejecutan en el propio objeto antes de persistir (ValidateGraph)
Datos flexibles tipo JSON	Parcial	IRIS maneja documentos, pero MongoDB ya cubre los esquemas flexibles por tenant en el TPO
Historial masivo de eventos	No	La escritura masiva append-only ya está resuelta con Cassandra
Caminos y relaciones del flujo	Parcial	Las relaciones se modelan, pero los recorridos de profundidad variable se consultan mejor como grafo
Caché y estado temporal	No	TTL y estructuras en memoria son de Redis
Integración políglota	Sí	El mismo modelo se accede por objetos y por SQL, lo que facilita convivir con el resto del stack